

Project Vision - Version 1.0.0 Rewrite

Objective

Rebuild the existing application from scratch using the current system as a business reference while eliminating technical debt, obsolete code, inconsistent patterns, and architectural mistakes accumulated over time.

The goal is not feature expansion. The goal is to create a clean, maintainable, extensible, production-grade foundation that can support future growth for the next several years.

High-Level Architecture

Architecture Style

- Modular Monolith
- Domain-Driven Design principles where appropriate
- Vertical Slice Architecture
- Clean boundaries between modules
- API-first approach
- Single deployable backend application
- Shared API consumed by:
 - React Web Application
 - Android Application
 - Future clients

Avoid microservices unless a clear business need emerges.

Technology Stack

Backend

- .NET 10
- ASP.NET Core Web API
- Minimal APIs
- Entity Framework Core
- SQL Server (or PostgreSQL)
- FluentValidation
- MediatR (or equivalent mediator)
- Serilog
- OpenTelemetry

- ProblemDetails
- Health Checks
- HybridCache
- Redis (when distributed cache is required)

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- TanStack Query
- React Hook Form
- Zod
- React Router

Mobile

- Existing Android application continues consuming the same backend API.
-

Core Principles

1. Domain First

Business rules belong inside the domain.

Avoid:

- Anemic domain models
- Business logic inside controllers
- Business logic inside UI

Prefer:

- Rich domain entities
 - Domain services when necessary
 - Explicit business invariants
-

2. Vertical Slice Architecture

Organize by feature, not by technical layer.

Example:

Modules/ Customers/ CreateCustomer/ UpdateCustomer/ DeleteCustomer/ GetCustomer/

Instead of:

Controllers/ Services/ Repositories/ DTOs/

Each feature owns:

- Request
 - Response
 - Validator
 - Handler
 - Endpoint
-

3. CQRS (Practical Version)

Commands modify state.

Queries read state.

Examples:

- CreateCustomerCommand
- UpdateCustomerCommand
- SearchCustomersQuery
- GetCustomerByIdQuery

Avoid:

- Event sourcing
- Separate databases
- Over-engineering

CQRS should improve clarity, not complexity.

4. API First

The API is the primary contract.

All clients consume the same API:

- React Web App

- Android App
- Future Integrations

Avoid creating special business logic for specific clients.

Data Access

Entity Framework Core

Use EF Core directly.

Avoid:

- Generic Repository Pattern
- Generic Unit of Work Pattern

DbContext already provides these abstractions.

Use repositories only when they provide actual business value.

Validation

All requests must be validated using FluentValidation.

Validation must occur before handlers execute.

Examples:

- Required fields
 - Length restrictions
 - Business constraints
 - Cross-field validation
-

Authentication

Use:

- ASP.NET Core Identity
- JWT Access Tokens
- Refresh Tokens

Support:

- Web
- Mobile

Future-ready for:

- SSO
 - External providers
-

Authorization

Use permission-based authorization.

Avoid role-only authorization.

Example:

Permissions:

- Customers.View
- Customers.Create
- Customers.Update
- Customers.Delete

Roles become collections of permissions.

Caching Strategy

Level 1

HybridCache

Use for:

- Reference data
- Configuration
- Frequently accessed read models

Level 2

Redis

Only when distributed caching is required.

Avoid caching everything.

Cache only after measuring bottlenecks.

Logging

Use structured logging.

Requirements:

- Correlation IDs
- Request tracing
- Exception logging
- Performance logging
- Audit logging where needed

Never log sensitive information.

Observability

Implement:

- OpenTelemetry
- Metrics
- Traces
- Health Checks

System health must be visible and measurable.

Error Handling

Use centralized exception handling.

Return consistent API responses.

Use ProblemDetails.

Avoid:

- Unhandled exceptions
 - Custom error formats per endpoint
-

Database Migrations

Use EF Core Migrations.

Workflow:

Development:

- Create migration
- Apply migration

Production:

- Generate SQL script
- Review script
- Backup database
- Apply migration

Never make manual schema changes.

Testing Strategy

Unit Tests

Test:

- Domain logic
- Business rules
- Validators

Integration Tests

Test:

- API endpoints
- Database interactions
- Authentication flows

Use Testcontainers where applicable.

Frontend Principles

UI Philosophy

Modern SaaS style.

Design goals:

- Clean
- Minimal
- Fast
- Responsive
- Accessible

Use:

- shadcn/ui components
- Consistent spacing system
- Consistent typography
- Consistent color system

Avoid random custom styling.

Frontend Architecture

Feature-based organization.

Example:

`src/features/customers/ components/ pages/ services/ hooks/ types/`

Avoid dumping everything into a shared components folder.

Development Rules

Always

- Follow SOLID principles
- Favor composition over inheritance

- Keep methods small
- Keep classes focused
- Use explicit naming
- Prefer readability over cleverness

Never

- Premature optimization
 - Unnecessary abstractions
 - Generic repositories
 - God services
 - Massive controllers
 - Massive handlers
 - Circular dependencies
-

Success Criteria

Version 1.0.0 should provide:

- Clean architecture
- Strong domain model
- Stable API contracts
- Modern React frontend
- Android compatibility
- Production-ready authentication
- Permission-based authorization
- Logging and observability
- Reliable migrations
- Automated testing
- Easy future extensibility

The primary goal is maintainability, clarity, and long-term evolution rather than implementing every new technology trend.